

CS 551 Systems Programming

Addendum: Borrowing fields, and lifetimes

Why this deck exists

Two things the main sequence leaves thin, and both are places where the borrow checker looks arbitrary until you see the rule.

Borrowing one field is not borrowing the struct. Lecture 7 showed you that a struct *moves* field by field. It never said that borrows work the same way, or where that stops working.

Lifetimes got one slide in Lecture 5 and a deferral in Lecture 7. That was the right call at the time: you do not want a lifetime parameter on your first struct. But the borrow checker's error messages are written in the language of lifetimes, so reading them means learning it.

Nothing here is new rules. It is the rules you have, stated precisely enough to predict what the compiler does.

Today's path

1. **Fields, not structs.** What the borrow checker actually tracks.
2. **The method boundary.** Why `&mut self` gives up the precision.
3. **Lifetimes.** What a lifetime is, and what it is not.
4. **Elision.** Why you have written almost none of them.
5. **Writing them down.** The cases where the compiler needs help.
6. **Structs that borrow.** What Lecture 7 deferred.

1. Fields, not structs

What the borrow checker actually tracks

You already saw this for moves

```
1 fn main() {  
2     let user1 = User {  
3         username: String::from("jeremy"),  
4         email: String::from("jblackbu@binghamton.edu"),  
5         sign_in_count: 1,  
6         active: true,  
7     };  
8     let user2 = User { email: String::from("other@example.edu"), ..user1 };  
9     println!("{}", user1.username);  
10 }
```

The error names the field

```
error[E0382]: borrow of moved value: `user1.username`
  8 |         let user2 = User { email: ..., ..user1 };
    |                               ----- value moved here
  9 |         println!("{}", user1.username);
    |                           ^^^^^^^^^^^^^ value borrowed here after move
```

Read the name. Not `user1`. `user1.username`. Ownership is tracked per field, and the compiler says so in the first line of the error.

"Partially moved" is a real state

```
1 struct Config { path: String, verbose: bool }
2 fn show(c: &Config) { println!("{}", c.path, c.verbose); }
3
4 fn main() {
5     let c = Config { path: String::from("/etc/hosts"), verbose: true };
6     let path = c.path;
7     show(&c);
8 }
```

Neither whole nor gone

```
error[E0382]: borrow of partially moved value: `c`
  6 |         let path = c.path;
    |             ----- value partially moved here
  7 |         show(&c);
    |             ^^ value borrowed here after partial move
= note: partial move occurs because `c.path` has type `String`,
       which does not implement the `Copy` trait
```

`path` moved on line 6. `verbose` did not. `c` is now **neither whole nor gone**, and `show(&c)` wants all of it.

`c.path.clone()` on line 6 leaves `c` whole, and the program compiles.

Borrows work the same way

```
1 struct Doc {
2     title: String,
3     lines: Vec<String>,
4 }
5
6 fn main() {
7     let mut doc = Doc { title: String::from("Notes"), lines: vec![] };
8     let t = &doc.title;
9     doc.lines.push(String::from("first line"));
10    println!("{t} {}", doc.lines[0]);
11 }
```

A shared borrow of `doc.title` and a mutable borrow of `doc.lines`, live at the same time, and it compiles.

These are two different places. Borrowing one says nothing about the other.

Borrows work the same way

```
1 struct Doc {
2     title: String,
3     lines: Vec<String>,
4 }
5
6 fn main() {
7     let mut doc = Doc { title: String::from("Notes"), lines: vec![] };
8     let t = &doc.title;
9     doc.lines.push(String::from("first line"));
10    println!("{t} {}", doc.lines[0]);
11 }
```

A different field is a different value

Mutable borrow of **lines**, while **title** is still borrowed

This is not an exception

Read the rule the way the compiler means it:

*At any point, for **any given value**, you may have many shared references or one mutable reference.*

The word doing the work is *value*. A struct is not one value as far as this rule is concerned. It is a field, and a field, and a field, and each of those is a value with its own borrows.

`doc.title` and `doc.lines` are disjoint values. No rule is being bent to let both borrows exist; they never overlapped in the first place.

A word on "value"

A value is usually a piece of memory, and thinking of it that way will not mislead you often. `doc.title` is a `String`: a length, a capacity, and a pointer to some bytes on the heap, sitting at its own address.

Copy values work the same way, they are just always copied. Ownership is not switched off for an `i32`. Every rule still applies: there is an owner, borrows are tracked, the borrow checker runs. The only difference is what an assignment or a call does to the original, and it copies rather than moves it, so the original is still there afterwards.

That is why `..user1` took `username` and left `sign_in_count` alone. Same rule, two fields, and one of them could be duplicated cheaply enough that Rust does it for you.

LIVE CODING CHALLENGE

The last slide compiled. Now move the push into a method on `Doc`, so the call reads `doc.add(...)` instead of `doc.lines.push(...)`.

Change nothing else.

Does it still compile, and what decided that?

Pause here. Try it before the next slide.

It does not

```
1 struct Doc {
2     title: String,
3     lines: Vec<String>,
4 }
5
6 impl Doc {
7     fn add(&mut self, line: String) {
8         self.lines.push(line);
9     }
10 }
11
12 fn main() {
13     let mut doc = Doc { title: String::from("Notes"), lines: vec![] };
14     let t = &doc.title;
15     doc.add(String::from("first line"));
16     println!("{t}");
17 }
```

The error names the struct

```
error[E0502]: cannot borrow `doc` as mutable because it is also borrowed as immutable
  |
14 |     let t = &doc.title;
  |           ----- immutable borrow occurs here
15 |     doc.add(String::from("first line"));
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
16 |     println!("{t}");
  |                   - immutable borrow later used here
```

doc, not **doc.lines**. The **..** error two sections ago named a field, **user1.username**. This one names the whole struct. That is the entire difference.

2. The method boundary

Where the precision stops

The signature is the whole story

Inside `add`, the body touches only `self.lines`. The compiler knows that. It does not use it.

Across a call, the only thing consulted is the signature. `fn add(&mut self, ...)` says: *I may touch all of it*. So at the call site, `doc.add(...)` takes a mutable borrow of the entire `Doc`, and the shared borrow held by `t` is still live.

This is deliberate, and it is the same reason function signatures are the contract everywhere else in Rust: changing a method's body must not break its callers. If borrow-checking looked through the body, adding one line inside `add` could break code that never mentions `add`.

You have met this shape before

Lecture 5 asked why the compiler cannot see that `v[0..2]` and `v[2..5]` are disjoint. The answer was that in general it cannot, and `split_at_mut` is what the language offers instead.

Same problem, one level up. The compiler can see disjointness when it can see the pieces: two named fields, two array halves handed back separately. It cannot see it through something whose signature says "all of it": an index expression, a method taking `&mut self`.

The fix has the same shape too. Hand back the pieces, or take only the pieces you need.

Four ways out

Do the field access inline. `doc.lines.push(...)` compiles. If the method adds nothing, it is costing you precision for free.

Take fields as parameters. A free function `fn add(lines: &mut Vec<String>, line: String)` borrows one field, and the caller does the splitting.

Split the struct. If two groups of fields are always used apart, that is a design signal: they may be two structs.

Reach for interior mutability, last. `RefCell<T>` moves the check to runtime. It is the right answer sometimes, and the wrong answer when you reached for it to avoid thinking about the first three.

Recap: part one

1. **Ownership and borrows are tracked per field**, not per struct.
2. **A method call is opaque.** Its signature decides what is borrowed, not its body.
3. **That is a feature.** Bodies stay free to change without breaking callers.
4. **When you need the precision, expose the pieces:** inline field access, or a function that takes just the field.

3. Lifetimes

What the borrow checker has been tracking all along

A lifetime is a region of code

Not a duration. Not a number of milliseconds. **A region of the program text** over which a reference is valid.

Every reference has one. You have written thousands and annotated almost none, because the compiler works them out.

The borrow checker's whole job, stated in these terms:

For every reference, prove that the thing it points at is still alive everywhere the reference is used.

A lifetime is how it writes that proof down. When it cannot finish the proof, the error message is written in the same vocabulary, which is why they are hard to read before you have it.

What it is protecting you from

```
1 fn dangle() -> &String {  
2     let s = String::from("hello");  
3     &s  
4 }
```

```
error[E0106]: missing lifetime specifier  
  |  
  |         ^ expected named lifetime parameter  
  = help: this function's return type contains a borrowed value, but there is  
         no value for it to be borrowed from  
help: instead, you are more likely to want to return an owned value  
1 + fn dangle() -> String {
```

`s` is dropped when `dangle` returns, so the reference would point at freed memory. In C this compiles and you find out later. **The compiler is asking: borrowed from what?**

The syntax, once

```
1 fn longest<'a>(x: &'a str, y: &'a str) -> &'a str
```

Read it as three claims:

- `<'a>` introduces a name for a region. It is a generic parameter, like `<T>`, except it ranges over regions instead of types.
- `x: &'a str`, `y: &'a str` say both inputs are valid for at least that region.
- `-> &'a str` says the output is valid for it too.

Together: *the returned reference is valid for as long as both inputs are.*

'a does not change how long anything lives. It states a relationship that already holds.

The one that needs it

```
1 let a = String::from("a longer string");
2 let b = String::from("short");
3 let winner = longest(&a, &b);
4 println!("{winner}");
```

Without the annotation this function does not compile, and the reason is worth stating precisely: **the compiler cannot tell which input the output came from.** It is not being lazy. `longest` returns `x` sometimes and `y` other times, and the answer depends on data it does not have.

So it asks you to promise something checkable instead: *the result is good for as long as the shorter-lived input.* That promise is `'a` on all three.

4. Elision

Why you have written almost none of these

Three rules, applied in order

You have written `fn first_word(s: &str) -> &str` many times. That is `fn first_word<'a>(s: &'a str) -> &'a str`, filled in mechanically.

1. **Every elided input lifetime gets its own parameter.** `fn f(a: &str, b: &str)` becomes `fn f<'a, 'b>(a: &'a str, b: &'b str)`.
2. **Exactly one input lifetime? It goes to every elided output.** This is why `first_word` needs nothing.
3. **A method with `&self`? `self`'s lifetime goes to every elided output.** This is why methods almost never need annotations.

If the rules run out and an output is still unfilled, you get `E0106`. `longest` has two inputs and no `self`, so neither rule 2 nor rule 3 fires.

Rule 3 is doing a lot of work

```
1 struct Report {  
2     lines: Vec<String>,  
3 }  
4  
5 impl Report {  
6     fn first(&self) -> &String {  
7         &self.lines[0]  
8     }  
9 }
```

`first` returns a reference **into** `self`. The full signature is `fn first<'a>(&'a self) -> &'a String`, and rule 3 wrote it.

The return value is not a copy. It is a window into the struct, and the borrow stays live for as long as you hold it.

Rule 3 is doing a lot of work

```
1 struct Report {  
2     lines: Vec<String>,  
3 }  
4  
5 impl Report {  
6     fn first(&self) -> &String {  
7         &self.lines[0]  
8     }  
9 }
```

Really `fn first<'a>(&'a self) -> &'a String`

5. Structs that borrow

What Lecture 7 deferred

Lecture 7's error, revisited

```
1 struct User {  
2     username: &str,  
3 }
```

```
error[E0106]: missing lifetime specifier  
2 |     username: &str,  
  |               ^ expected named lifetime parameter  
help: consider introducing a named lifetime parameter  
1 ~ struct User<'a> {  
2 ~     username: &'a str,
```

The question is the same one `dangle` could not answer: **borrowed from what?**

A `User` holding a `&str` is only valid while the text it points at is alive. The struct has to say so, because anyone holding a `User` needs to know.

Saying it

```
1 struct User<'a> {  
2     username: &'a str,  
3 }  
4  
5 fn main() {  
6     let name = String::from("jeremy");  
7     let u = User { username: &name };  
8     println!("{}", u.username);  
9 }
```

`User<'a>` reads: a `User` that may not outlive the string it borrows. The compiler now checks that at every use.

Swap the two lines in `main` so `name` is dropped first, and this stops compiling. That is the whole point: the annotation turned a runtime disaster into a compile error.

Saying it

```
1 struct User<'a> {  
2     username: &'a str,  
3 }  
4  
5 fn main() {  
6     let name = String::from("jeremy");  
7     let u = User { username: &name };  
8     println!("{}", u.username);  
9 }
```

Names a region. A parameter, like <T>

This is the borrow 'a was naming

Why Lecture 7 still says own your fields

The annotation works. It also spreads.

Every function taking a `User<'a>` carries the parameter. Every struct holding one carries it. You end up threading `'a` through half the program to avoid one allocation, and the first time you need to store a `User` somewhere long-lived, you unpick all of it.

Own your fields by default. `String`, not `&'a str`. Take the allocation.

Borrow in a struct when the struct is a short-lived view over data someone else owns: a parser over a buffer, an iterator over a collection, a formatter over a value. Those are real and common, and they are all cases where the struct is gone before the data is.

'static' is not a magic word

'static' means *valid for the whole program*. String literals are `&'static str`, because they live in the binary.

When a compiler error says a reference needs to be `'static`, the fix is almost never to write `'static`. That is agreeing to a promise you cannot keep, and the error moves rather than going away.

The fix is usually to own the data instead. `String` rather than `&'static str`, a `Vec<T>` rather than a borrowed slice. Reach for `'static` when the data genuinely is static, not to quiet a message.

Reading the errors

Error	What it is asking
<code>E0106</code> missing lifetime specifier	Borrowed from what? Name the source.
<code>E0597</code> borrowed value does not live long enough	The source dies before the reference is done.
<code>E0502</code> cannot borrow as mutable	A shared borrow is still live. Look for a method taking <code>&mut self</code> .
<code>E0499</code> cannot borrow as mutable more than once	Two <code>&mut</code> to the same place. Look for disjoint fields.
<code>E0382</code> borrow of moved value	It moved. Look for a <code>..</code> , a <code>self</code> method, or a plain assignment.

Recap: part two

1. **A lifetime is a region of code**, not a duration, and every reference has one.
2. **Annotations state relationships; they do not change them.** `'a` cannot make anything live longer.
3. **Elision writes most of them for you**, and rule 3 is why methods returning references tie the result to `&self`.
4. **A struct holding a reference needs a parameter**, and that parameter spreads. Own your fields unless the struct is a view.
5. **The error messages are written in this vocabulary.** That was the real reason to learn it.

Exercises

1. Take the `Doc` example that failed and make it compile three different ways: inline field access, a free function over one field, and splitting the struct. Which reads best?
2. Write `fn longest` without the annotations and read the error. Then add only `<'a>` and read the next one.
3. Give `Report::first` an explicit signature that spells out what rule 3 inferred. Confirm it still compiles.
4. Build a `User<'a>` in an inner block and try to use it outside. Match the error to the table above.

Reading

- **The Rust Programming Language**, chapter 10.3 (Validating References with Lifetimes) and chapter 4.2 on references.
- **The Rustonomicon**, the lifetimes chapter, when you want the precise version and are ready for it.
- **The Rust Reference**, on lifetime elision, which states the three rules as the compiler implements them.